

[Lecture Note] Decision Structures and Boolean Logic

MGS3001 Python Programming for Business, Spring 2025

Chad (Chungil Chae)

Sun, 9 March 2025

Learning Objectives

(AUTHOR, 9999).

SHORT VIDEO INTRODUCTION

<https://www.youtube.com/watch?v=K5ea4-Omu-I>

The if statement

A control structure is a logical design that controls the order in which a set of statements execute. So far in this book, we have used only the simplest type of control structure: **the sequence structure**.

- A sequence structure is a set of statements that execute in the order in which they appear.

```
name = input('What is your name? ')
age = int(input('What is your age? '))
print('Here is the data you entered:')
print('Name:', name)
print('Age:', age)
```

①
②
③
④
⑤

- ① Get name from user input
- ② Get age as integer from user input
- ③ Display a message
- ④ Display name
- ⑤ Display age

- Some problems simply cannot be solved by performing a set of ordered steps, one after the other.
 - e.g. A pay calculating program that determines whether an employee has worked overtime. If the employee has worked more than 40 hours, he or she gets paid extra for all the hours over 40. Otherwise, the overtime calculation should be skipped.

Programs like this require a different type of control structure: one that can execute a set of statements only under certain circumstances. This can be accomplished with a **decision structure**. (Decision structures are also known as selection structures.)

- In a decision structure's simplest form, a specific action is performed only if a certain condition exists.
- If the condition does not exist, the action is not performed. The flowchart shows how the logic of an everyday decision can be diagrammed as a decision structure.
- The diamond symbol represents a true/false condition.
 - If the condition is true, we follow one path, which leads to an action being performed.
 - If the condition is false, we follow another path, which skips the action.
- In the flowchart the diamond symbol indicates some condition that must be tested.
 - In this case, we are determining whether the condition Cold outside is true or false.
 - * If this condition is true, the action Wear a coat is performed.
 - * If the condition is false, the action is skipped.

The action is conditionally executed because it is performed **only when a certain condition is true**.

- Programmers call the type of **decision structure** shown as a single alternative decision structure.
- This is because it provides only one alternative path of execution.
 - If the condition in the diamond symbol is true, we take the alternative path.
 - Otherwise, we exit the structure.

if statement to write a single alternative decision structure. Here is the general format of the if statement:

```
if condition: ①  
    statement ②  
    statement  
    etc
```

① The first line as the if clause.

② Sequence structure

- The if clause begins with the word **if**, followed by a condition, which is an expression that will be evaluated as either true or false.
- A **colon (:)** appears after the condition.
- Beginning at the next line is a block of statements.

- A block is simply a set of statements that belong together as a group.
- Notice in the general format that all of the statements in the block are indented.
 - This indentation is required because the Python interpreter uses it to tell where the block begins and ends.
- When the if statement executes, the condition is tested.
 - If the condition is true, the statements that appear in the block following the if clause are executed.
 - If the condition is false, the statements in the block are skipped.

Boolean Expressions and Relational Operators

- The if statement tests an expression to determine whether it is true or false.
- The expressions that are tested by the if statement are called Boolean expressions, named in honor of the English mathematician George Boole.
- In the 1800s, Boole invented a system of mathematics in which the abstract concepts of true and false can be used in computations.
- Typically, the Boolean expression that is tested by an if statement is formed with a relational operator. A relational operator determines whether a specific relationship exists between two values.
- For example, the greater than operator ($>$) determines whether one value is greater than another. The equal to operator ($==$) determines whether two values are equal.

Relational operators

Operator	Meaning
$>$	Greater than
$<$	Less than
$>=$	Greater than or equal to
$<=$	Less than or equal to
$==$	Equal to
$!=$	Not equal to

Boolean expressions using relational operators

Operator	Meaning
$x > y$	Is x Greater than y?
$x < y$	Is x Less than y?
$x >= y$	Is x Greater than or equal to y?
$x <= y$	Is x Less than or equal to y?
$x == y$	Is x Equal to y?
$x != y$	Is x Not equal to y?

70

```
x = 1
y = 0
x > y
```

```
x = 1
y = 0
y > x
```

```
x = 1
y = 0
z = 1
x >= y
x <= z
x <= y
```

```
x = 1
y = 0
z = 1
x >= y
x <= z
x <= y
```

```
x = 1
y = 0
z = 1
x == y
x == z
y == z
```

```
x = 1
y = 0
z = 1
x != y
x != z
```

71 Putting It All Together

72 This statement uses the `>` operator to determine whether sales is greater than 50,000. If the expression sales
73 `> 50000` is true, the variable `bonus` is assigned 500.0. If the expression is false, however, the assignment
74 statement is skipped.

75 Flowchart for this section of code is the following

76 The following example conditionally executes a block containing three statements

77 The following code uses the `==` operator to determine whether two values are equal. The expression `balance == 0` will be true if the `balance` variable is assigned 0. Otherwise, the expression will be false.

```
if balance == 0: # Statements appearing here will be executed only if balance is equal to 0.
```

79 The following code uses the `!=` operator to determine whether two values are not equal. The expression `choice != 5` will be true if the `choice` variable does not reference the value 5. Otherwise, the expression will be false.

```
if balance != 5: # Statements appearing here will be executed only if balance is not equal to 5.
```

82 The if-else Statement

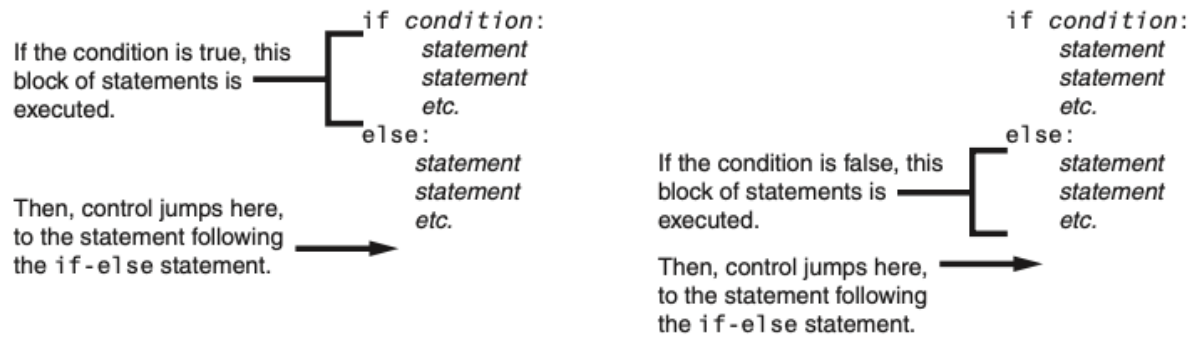
83 The dual alternative decision structure, which has two possible paths of execution—one path is taken if a condition is true, and the other path is taken if the condition is false.

85 In code, we write a dual alternative decision structure as an if-else statement. Here is the general format of the if-else statement:

```
if condition:
    statement
    statement
    etc.
else:
    statement
    statement
    etc.
```

87 When this statement executes, the condition is tested. If it is true, the block of indented statements following the if clause is executed, then control of the program jumps to the statement that follows the if-else statement.

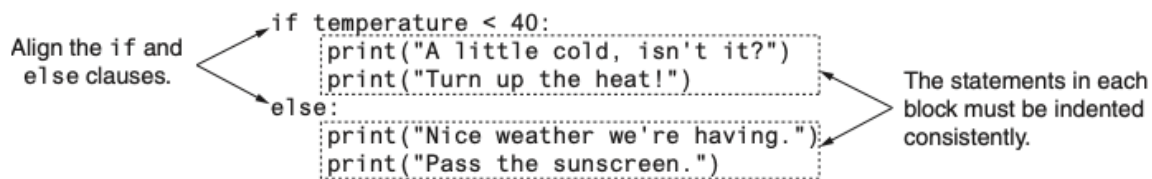
89 If the condition is false, the block of indented statements following the else clause is executed, then control of the program jumps to the statement that follows the if-else statement.



91

92 Indentation in the if-else statement

- 93 • When you write an if-else statement, follow these guidelines for indentation:
 - 94 – Make sure the if clause and the else clause are aligned.
 - 95 – The if clause and the else clause are each followed by a block of statements. Make sure the
 - 96 statements in the blocks are consistently indented.



97

98

99 Comparing Strings

100 Python allows you to compare strings. This allows you to create decision structures that test
 101 the value of a string

102

103 You saw in the preceding examples how numbers can be compared in a decision structure. You can also
 104 compare strings. For example, look at the following code:

```
name1 = 'Mary'
name2 = 'Mark'
if name1 == name2:
    print('The names are the same.')
else:
    print('The names are NOT the same.')
```

105 The == operator compares name1 and name2 to determine whether they are equal. Because the strings
106 'Mary' and 'Mark' are not equal, the else clause will display the message 'The names are NOT the same.'

107 Let's look at another example. Assume the month variable references a string. The following code uses the
108 != operator to determine whether the value referenced by month is not equal to 'October':

```
if month != 'October':
    print('This is the wrong time for Octoberfest!')
```

109 Other String Comparisons

110 In addition to determining whether strings are equal or not equal, you can also determine whether one string
111 is greater than or less than another string. This is a useful capability because programmers commonly need
112 to design programs that sort strings in some order.

113 Computer store numeric codes that represent the characters. Chapter 1 mentioned that ASCII (the American
114 Standard Code for Information Interchange) is a commonly used character coding system. You can see the
115 set of ASCII codes in Appendix C, but here are some facts about it:

116 The uppercase characters A through Z are represented by the numbers 65 through 90. The lowercase char-
117 acters a through z are represented by the numbers 97 through 122. When the digits 0 through 9 are stored
118 in memory as characters, they are represented by the numbers 48 through 57. (For example, the string
119 'abc123' would be stored in memory as the codes 97, 98, 99, 49, 50, and 51.)

120 A blank space is represented by the number 32. In addition to establishing a set of numeric codes to represent
121 characters in memory, ASCII also establishes an order for characters. The character "A" comes before the
122 character "B", which comes before the character "C", and so on.

123 When a program compares characters, it actually compares the codes for the characters.

124 For example, look at the following if statement:

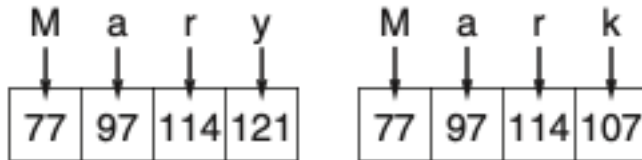
```
if 'a' < 'b':
    print('The letter a is less than the letter b.')
```

125 This code determines whether the ASCII code for the character 'a' is less than the ASCII code for the
126 character 'b'. The expression 'a' < 'b' is true because the code for 'a' is less than the code for 'b'. So, if
127 this were part of an actual program it would display the message 'The letter a is less than the letter b.'

Let's look at how strings containing more than one character are typically compared. Suppose a program uses the strings 'Mary' and 'Mark' as follows:

```
name1 = 'Mary'
name2 = 'Mark'
```

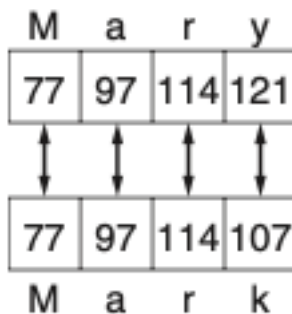
The following shows how the individual characters in the strings 'Mary' and 'Mark' would actually be stored in memory, using ASCII codes.



When you use relational operators to compare these strings, the strings are compared character-by-character. For example, look at the following code:

```
name1 = 'Mary'
name2 = 'Mark'
if name1 > name2:
    print('Mary is greater than Mark')
else:
    print('Mary is not greater than Mark')
```

The > operator compares each character in the strings 'Mary' and 'Mark', beginning with the first, or leftmost, characters.



Here is how the comparison takes place: 1. The 'M' in 'Mary' is compared with the 'M' in 'Mark'. Since these are the same, the next characters are compared. 2. The 'a' in 'Mary' is compared with the 'a' in 'Mark'. Since these are the same, the next characters are compared. 3. The 'r' in 'Mary' is compared with the 'r' in 'Mark'. Since these are the same, the next characters are compared. 4. The 'y' in 'Mary' is compared with the 'k' in 'Mark'. Since these are not the same, the two strings are not equal. The character 'y' has a higher ASCII code (121) than 'k' (107), so it is determined that the string 'Mary' is greater than the string 'Mark'.

If one of the strings in a comparison is shorter than the other, only the corresponding characters will be compared. If the corresponding characters are identical, then the shorter string is considered less than the longer string. For example, suppose the strings 'High' and 'Hi' were being compared. The string 'Hi' would be considered less than 'High' because it is shorter.

Nested Decision Structures and the if-elif-else Statement

In Section 3.1, we mentioned that a control structure determines the order in which a set of statements execute. Programs are usually designed as combinations of different control structures. For example, Figure 3-10 shows a flowchart that combines a decision structure with two sequence structures.

The flowchart in the figure starts with a sequence structure. Assuming you have an out- door thermometer in your window, the first step is Go to the window, and the next step is Read thermometer. A decision structure appears next, testing the condition Cold outside. If this is true, the action Wear a coat is performed. Another sequence structure appears next. The step Open the door is performed, followed by Go outside. Quite often, structures must be nested inside other structures. For example, look at the partial flowchart in Figure 3-11. It shows a decision structure with a sequence structure nested inside it. The decision structure tests the condition Cold outside. If that condition is true, the steps in the sequence structure are executed.

You can also nest decision structures inside other decision structures. In fact, this is a common requirement in programs that need to test more than one condition. For example, consider a program that determines whether a bank customer qualifies for a loan. To qualify, two conditions must exist: (1) the customer must earn at least \$30,000 per year, and (2) the customer must have been employed for at least two years. Figure 3-12 shows a flowchart for an algorithm that could be used in such a program. Assume the salary variable is assigned the customer's annual salary, and the years_ on_ job variable is assigned the number of years that the customer has worked on his or her current job.

If we follow the flow of execution, we see that the condition salary >= 30000 is tested. If this condition is false, there is no need to perform further tests; we know the customer does not qualify for the loan. If the condition is true, however, we need to test the second condition. This is done with a nested decision structure that tests the condition years_ on_ job >= 2. If this condition is true, then the customer qualifies for the loan. If this condition is false, then the customer does not qualify. Program 3-5 shows the code for the complete program.

Look at the if-else statement that begins in line 15. It tests the condition salary >= MIN_SALARY. If this condition is true, the if-else statement that begins in line 16 is executed. Otherwise the program jumps to the else clause in line 22 and executes the statement in lines 23 through 25. It's important to use proper indentation in a nested decision structure. Not only is proper indentation required by the Python interpreter, but it also makes it easier for you, the human reader of your code, to see which actions are performed by each part of the structure. Follow these rules when writing nested if statements:

- Make sure each else clause is aligned with its matching if clause. This is shown in Figure 3-13.
- Make sure the statements in each block are consistently indented. The shaded parts of Figure 3-14 show the nested blocks in the decision structure. Notice each statement in each block is indented the same amount.

Testing a series of conditions

In the previous example, you saw how a program can use nested decision structures to test more than one condition. It is not uncommon for a program to have a series of conditions to test, then perform an action depending on which condition is true. One way to accomplish this is to have a decision structure with numerous other decision structures nested inside it. For example, consider the program presented in the following In the Spotlight section.

The if-elif-else Statement

Logical Operators

Boolean Variables

Turtle Graphics: Determining the State of the Turtle

Reference

AUTHOR. (9999). *TITLE*. THE CHAD.